MIGRATING TO FABRIC: STORIES FROM THE FIELD

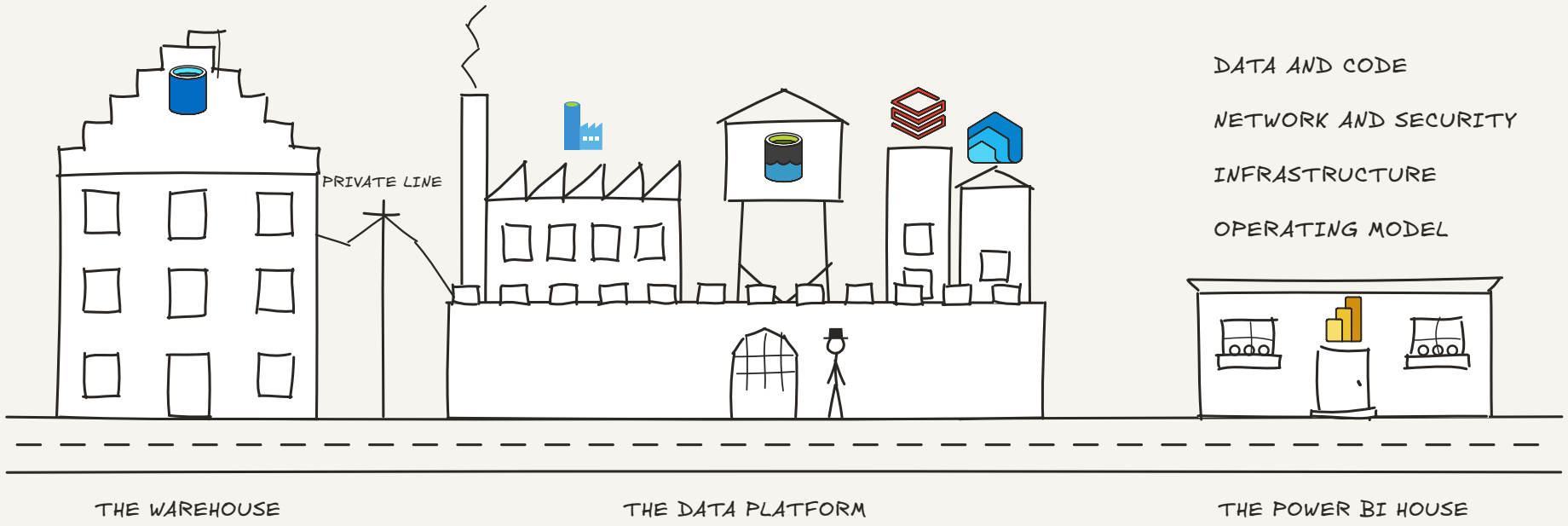
Don't simply move to Fabric. Design first.

Daan Humblé | Cloud Solution Architect

Stories from the field for enterprise architects planning a migration to Microsoft Fabric. The talk is about the architectural and organisational decisions a migration forces, not about product features.

FROM THE FIELD

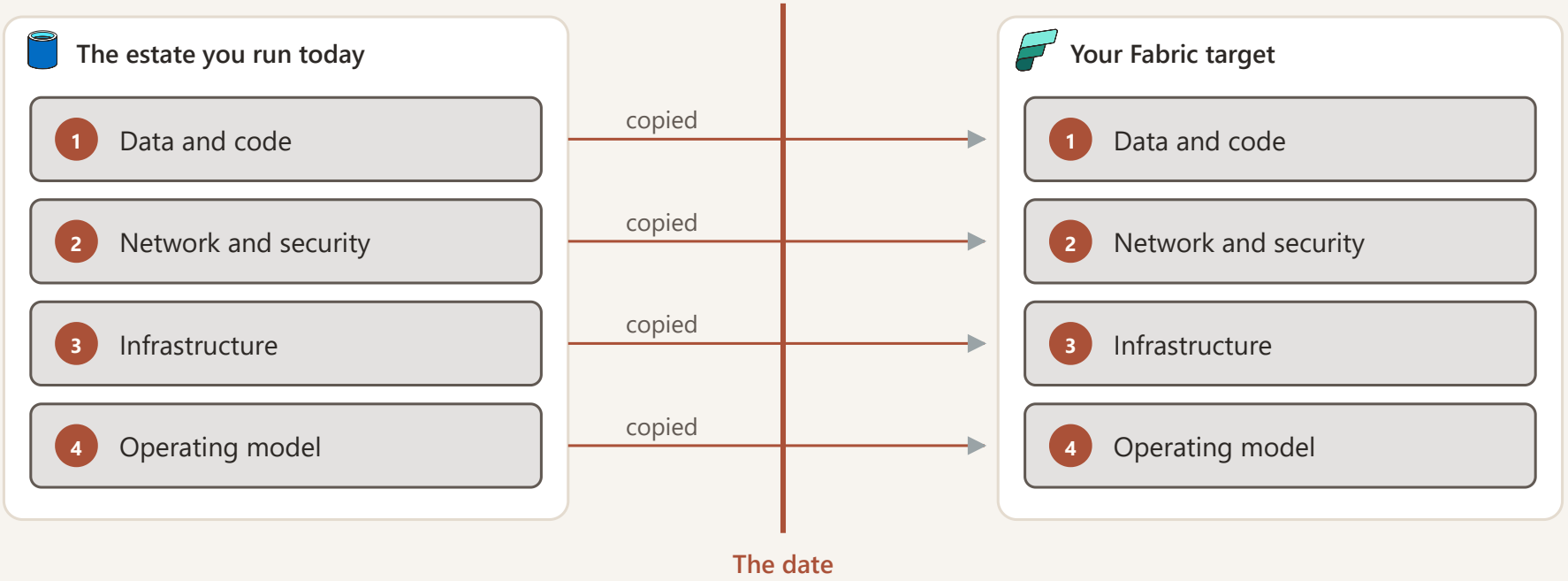
You already have an estate



An existing data estate, whether one building or a whole city, has four layers: data and code (the buildings), network and security (walls, gates, private lines), infrastructure (the street) and the operating model (the caretaker). Each was built for a reason, mostly unwritten.

NOW YOU ARE BUILDING A NEW ESTATE

How do you approach it?



Your legacy estate became your target architecture.

Moving to Fabric means building a new platform. Copying it over, often under a deadline, means each layer of the existing estate, from data and code to security, infrastructure boundaries and the operating model, is carried as it is, and the legacy estate becomes the target architecture for Fabric by default rather than by decision.

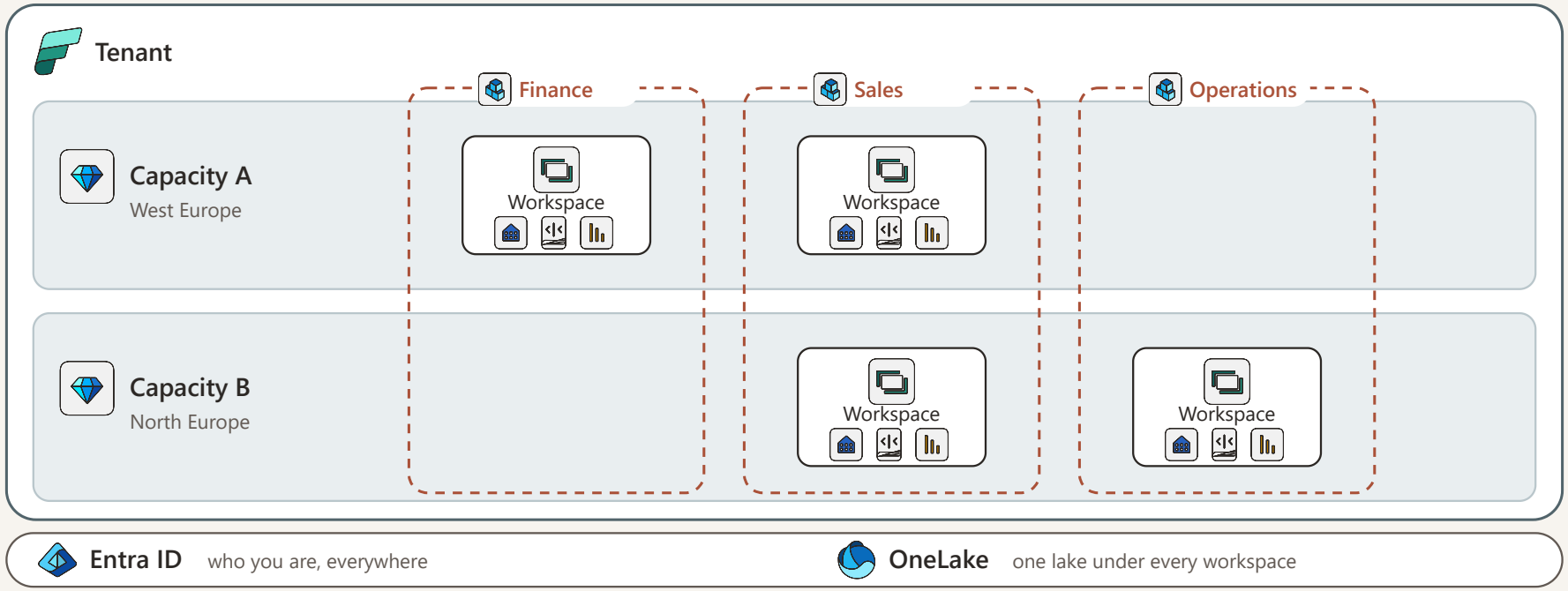
Your old estate drew your boundaries for you.

Now draw them on purpose.

Existing estates encode their boundaries in their four layers: data and code, network and security, infrastructure and the operating model. Fabric is a shared SaaS platform where those boundaries are configured rather than given. A migration that does not remake those decisions explicitly inherits the platform defaults.

THE NEW ESTATE

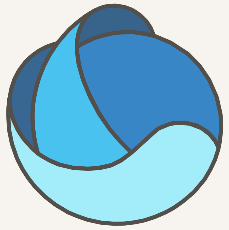
One tenant, workspaces, capacities and domains



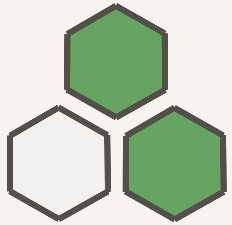
A Fabric tenant follows Microsoft Entra ID and has one OneLake underneath all its workspaces. Work happens in items such as warehouses, notebooks and reports, which live in workspaces. Capacities provide compute in a region and are funded as a unit. Domains group workspaces by ownership. Each workspace is assigned to one capacity and at most one domain, and these are independent choices.

NOT TODAY

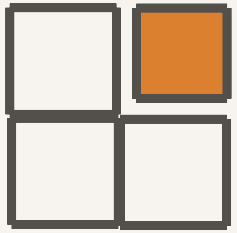
Fabric is amazing. You don't need to hear that every day



OneLake



Agentic Engineering



Fabric Apps

What customers run into as they adopt Fabric, and why.

This session leaves out the platform features, such as OneLake, agentic data engineering and applications built on Fabric. It covers patterns seen in the field as organisations migrate to and implement Fabric, whether as a single cut-over or gradually.

THE MAP

Four stories, four principles

DATA AND CODE

Quality

The house

NETWORK AND SECURITY

Security

The lock

INFRASTRUCTURE

Scalability

The bar

OPERATING MODEL

Governance

The caretaker

A modern data estate starts from business drivers, not products: one version of the truth, fewer silos, and data that is findable and shareable. Four architecture principles deliver them, one per layer of the existing estate: quality for data and code, security for network and security, scalability for infrastructure, and governance for the operating model. The talk tells one field story per principle: a house moving, a lock, a bar with one tap, and a caretaker. A platform such as Fabric is one way to realise these principles, not the reason to move.

WHO IS TALKING

Daan Humblé



Cloud Solution Architect · Microsoft

LinkedIn /in/daanhumble
GitHub /HumbleDaan

Helping organizations in adopting Fabric

Presented by Daan Humblé, Cloud Solution Architect at Microsoft. LinkedIn: /in/daanhumble. GitHub: /HumbleDaan. The stories come from helping customers, large and small, take their first steps into Fabric.

A NOTE ON AI

AI helped me build what you are seeing

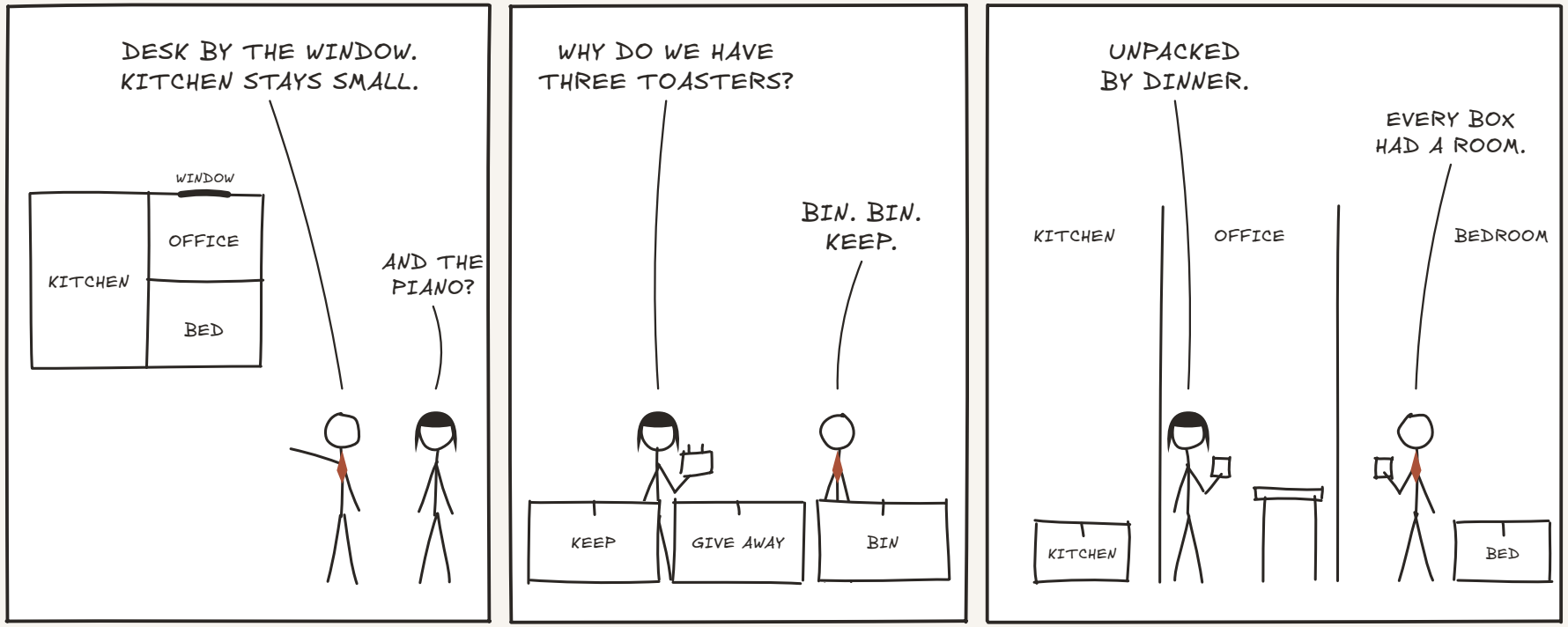
The stories are mine

Inspired by xkcd, by Randall Munroe

AI tools helped produce this deck. The stories and lessons are the speaker's own, from customer work. The drawing style is inspired by xkcd by Randall Munroe.

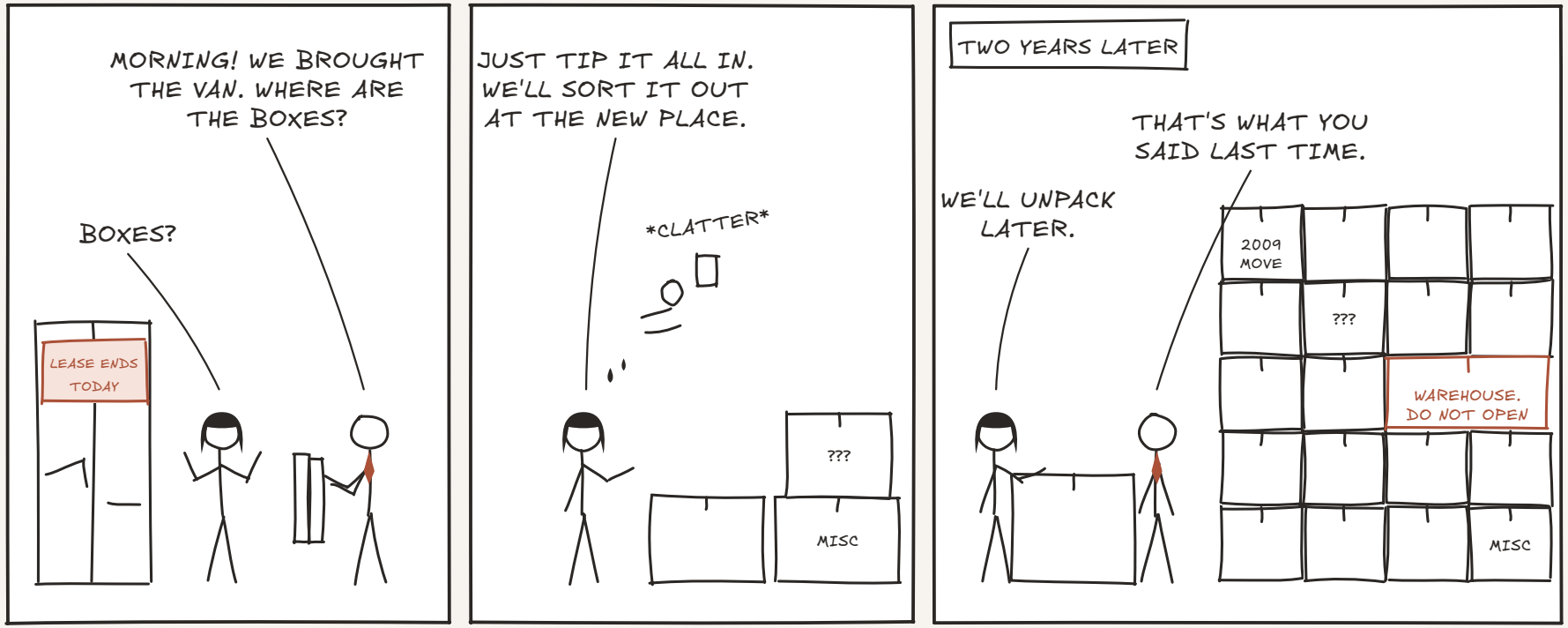
1 The house 2 The lock 3 The bar 4 The caretaker

The first story is about moving: what an existing estate takes with it into Fabric.



PLAN THE NEW HOUSE. THEN PACK FOR IT.

A well-run house move starts with the new house: decide where things go, sort what to keep, give away or throw out, and pack each box for a room. Designing for Fabric works the same way: plan the target, decide what not to migrate, then move.

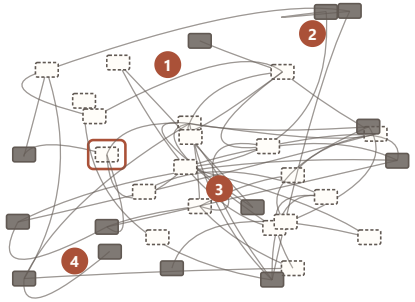


PACK IT ALL, SORT IT OUT LATER. LATER NEVER COMES.

A move driven by the end of a lease: nothing is planned, everything goes into boxes unsorted and the new house fills with boxes that never get unpacked. A migration done against a date is the same: the estate is copied as is, and the clean-up that was meant to follow rarely happens.

Over time, everything grows together

The warehouse you run today



1. **Recompute, don't store**
   ~~Storage was scarce; compute was already paid for~~
   Storage is cheap; every recompute is billed
2. **Someone decides who waits**
   ~~A workload manager queues the queries~~
   One shared capacity; everyone slows together
3. **Tuned for one engine**
   ~~Views on views, shaped for one optimiser~~
   A different engine; the tuning does not travel
4. **The night is for batch**
   ~~ETL owns the hours nobody works~~
   No quiet night; batch shares the day's capacity

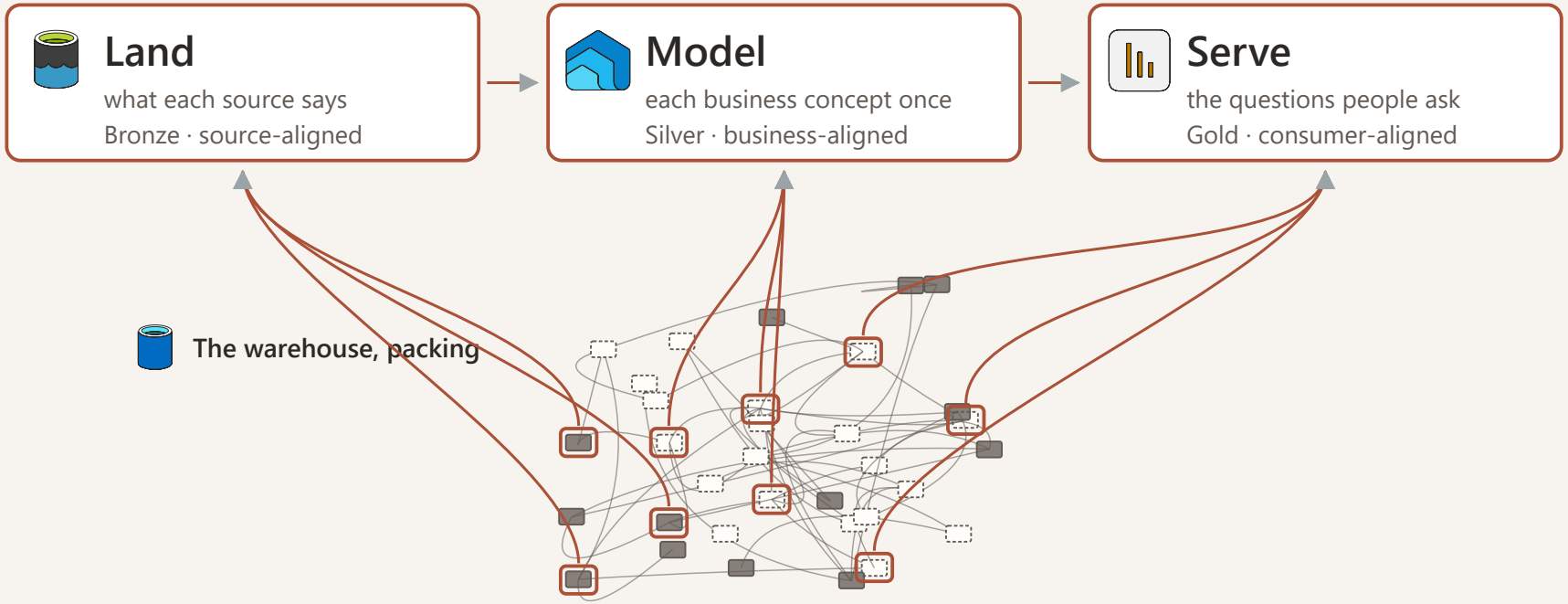
Fabric

SPAGHETTI IN, SPAGHETTI OUT

Moving this to Fabric

A long-lived on-premises warehouse encodes implicit decisions: the on-premises economics of scarce storage and sunk-cost compute, which favour recomputing over storing; workload management, engine-specific tuning and batch windows. Migrated unchanged to Fabric, the same logic runs where the economics are reversed, cheap storage and metered compute, so those decisions no longer hold, which typically makes it slower and more expensive.

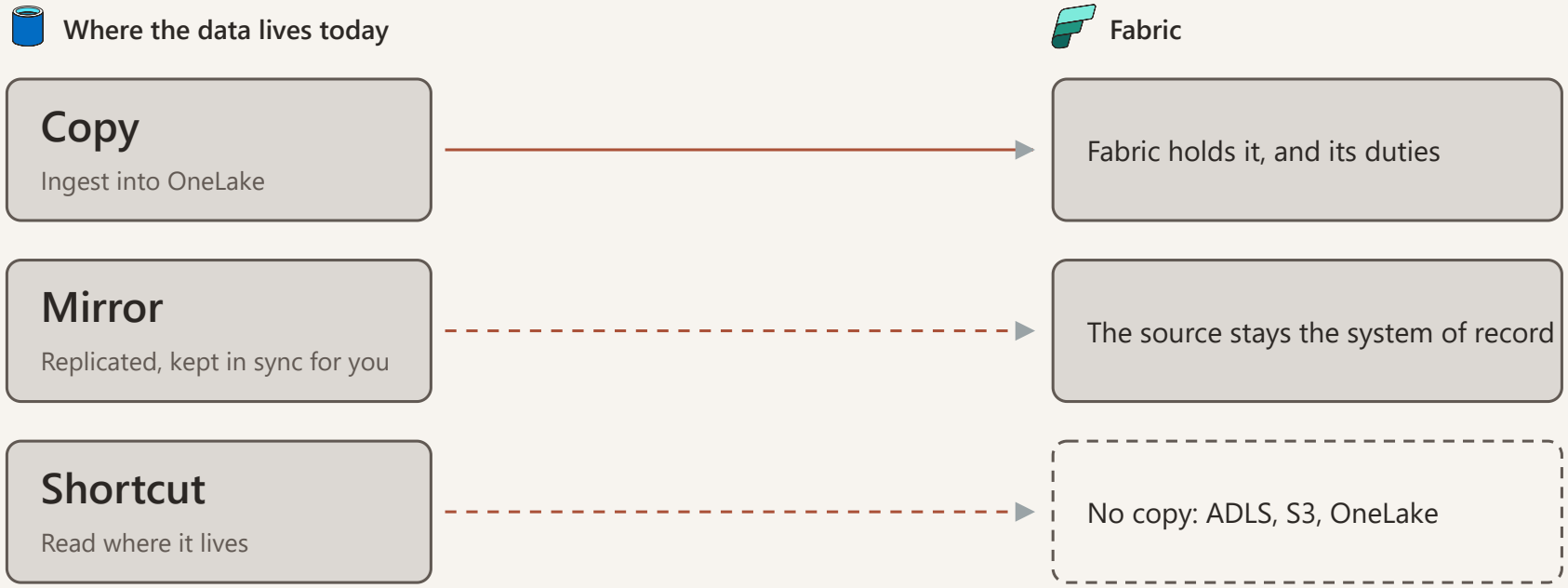
Label every box. Ship it to the right room directly.



Quality: one purpose, one owner, one room.

Migrating an entangled warehouse is the moment to sort its objects by purpose rather than copy them: source data lands as received (source-aligned), each business concept is modelled once (business-aligned), and serving objects answer specific questions (consumer-aligned); in medallion terms bronze, silver and gold, with lineage running one way. Labelled stages answer what the tangle could not: each has an owner, a break reaches only what is downstream, and a change has one place to go.

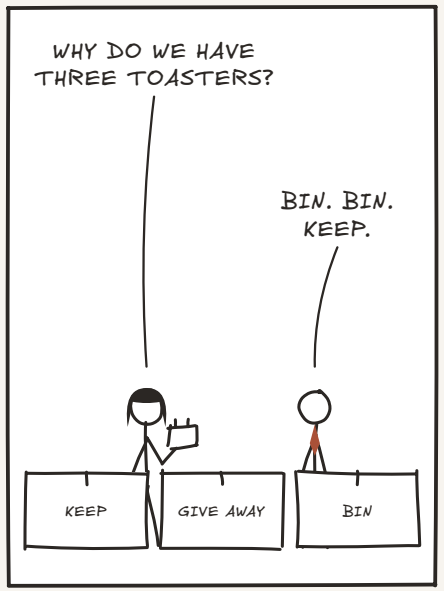
Not every box has to move



Connect, don't copy. Mix them: hybrid is a design, not a compromise.

Data does not have to be copied into Fabric to be used there. It can be ingested into OneLake, mirrored from a source database that remains the system of record, or reached through a shortcut that reads it in place (for example in ADLS Gen2, Amazon S3 or another workspace). Each copy is a new silo whose holder inherits the owner's security, privacy and legal obligations, so connecting is preferred where it suffices, and hybrid estates combining all three are a legitimate target.

Keep it, bin it, or optimise it



For every object in the warehouse:

What was it for on the old platform?

Is that still needed?

KEEP IT

Still needed,
fits the new engine

e.g. customer master

BIN IT

Nobody needs it
any more

e.g. the 2009 report view

OPTIMISE IT

Still needed, built
for the old engine

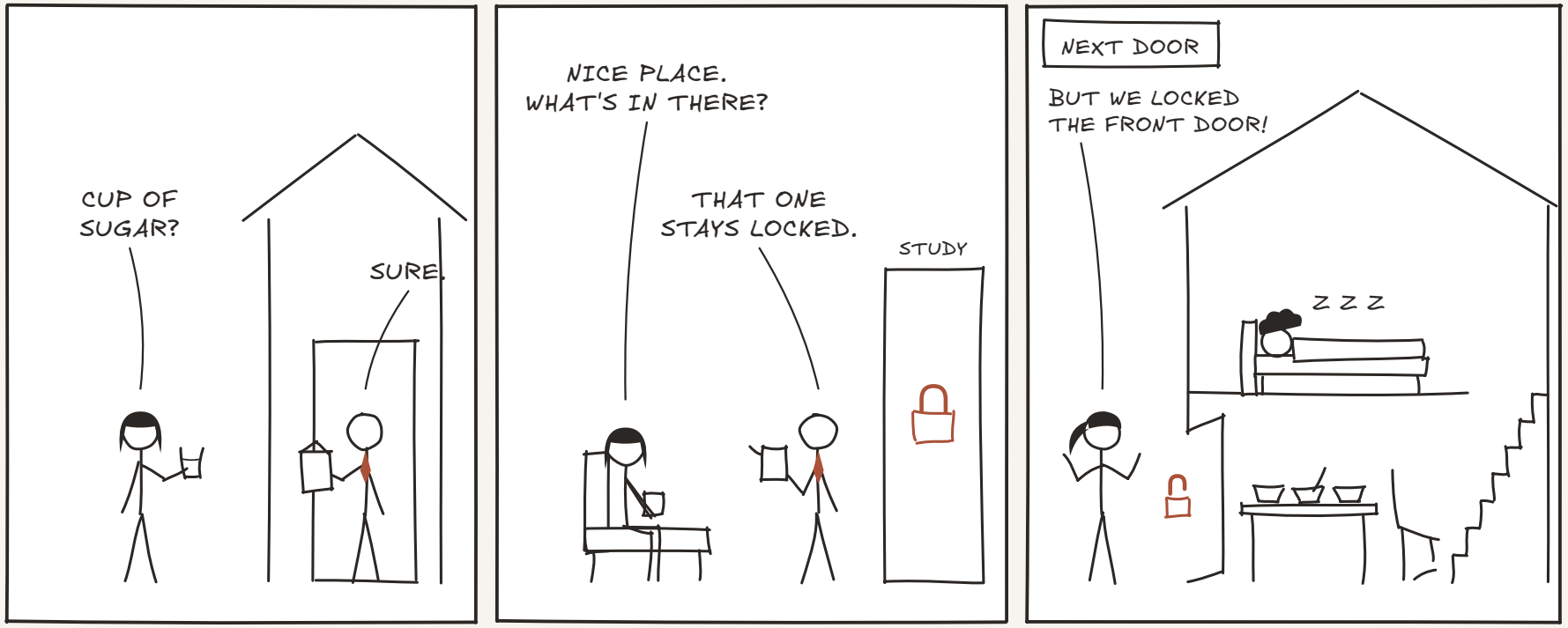
e.g. nightly recompute: store it

Inventory the reason, not just the object.

An existing estate encodes decisions implicitly: what is isolated, who owns, who pays and what fails together. Consolidating onto a shared platform removes the infrastructure that enforced those decisions. Make them explicit in the inventory, then decide which still apply.

Example: The sorting panel from the moving strip: before packing, every object gets a verdict. In the warehouse the verdict

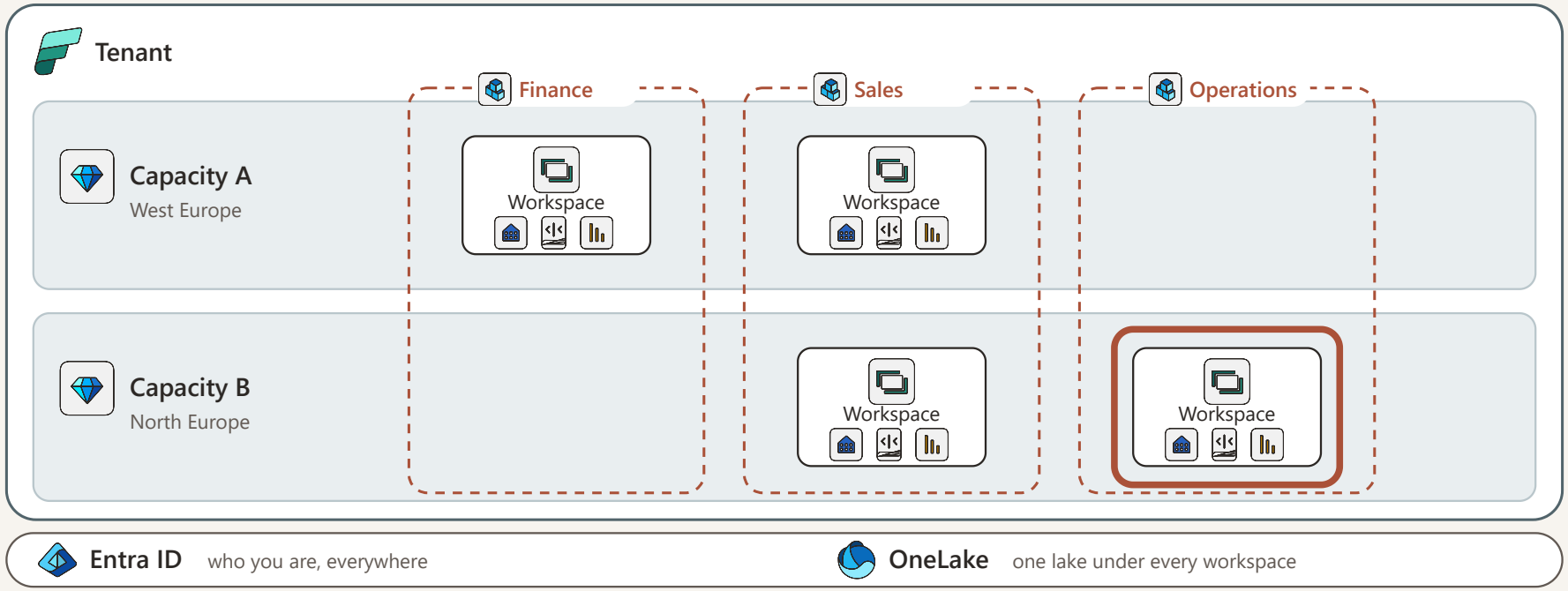
Recap of The house, delivering the principle of quality: 1.1 A one-to-one copy keeps the topology and loses the intent. 1.2 Align with the engine. Structure the estate while you do. 1.3 Inventory the decisions, not just the workloads.



TRUST BY THE ROOM, NOT BY THE FRONT DOOR.

A neighbour borrowing sugar needs no background check, and a neighbour having coffee in the living room still does not get into the study. A house with only a front-door lock has nothing left to protect it once that lock is passed. Fabric security works the same way: graded by what is at risk, with the strongest protection on the few items that need it, rather than one strict lock on everything or one loose lock in front of everything.

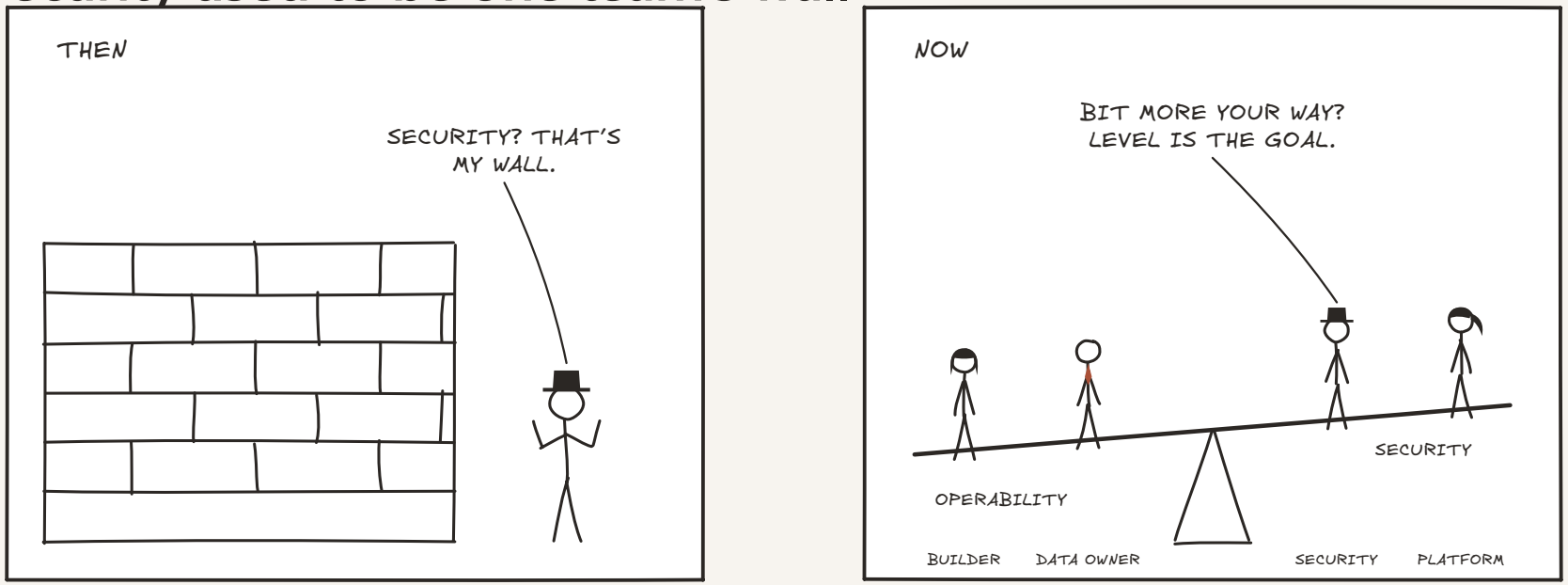
Security lives in the rooms



Workspace, item, data and down: the room and the closet.
Set it higher up and you will not be happy.

In Fabric, access is set at workspace level, item level and inside the data (row-, column- and object-level security). Tenant and domain settings are too coarse to protect specific data and should not be relied on for it.

Security used to be one team's wall

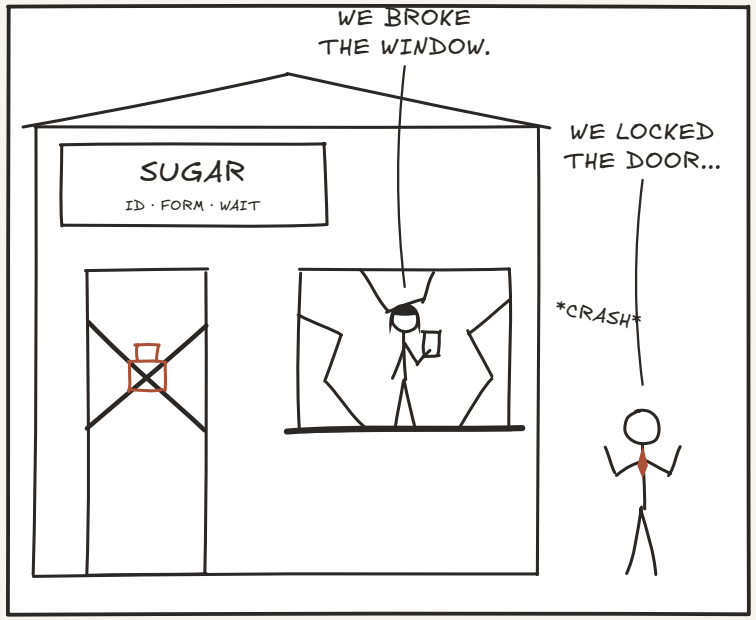


Shared responsibility: tune the balance, don't max it out.

Under the cloud shared responsibility model, a SaaS provider operates the infrastructure while data, accounts and identities remain the customer's responsibility. Inside the organisation, that responsibility is shared between security, platform teams, data owners and the people building on the data. Security becomes a balance between operability and protection rather than a single perimeter set once.

Lock the kitchen like the study



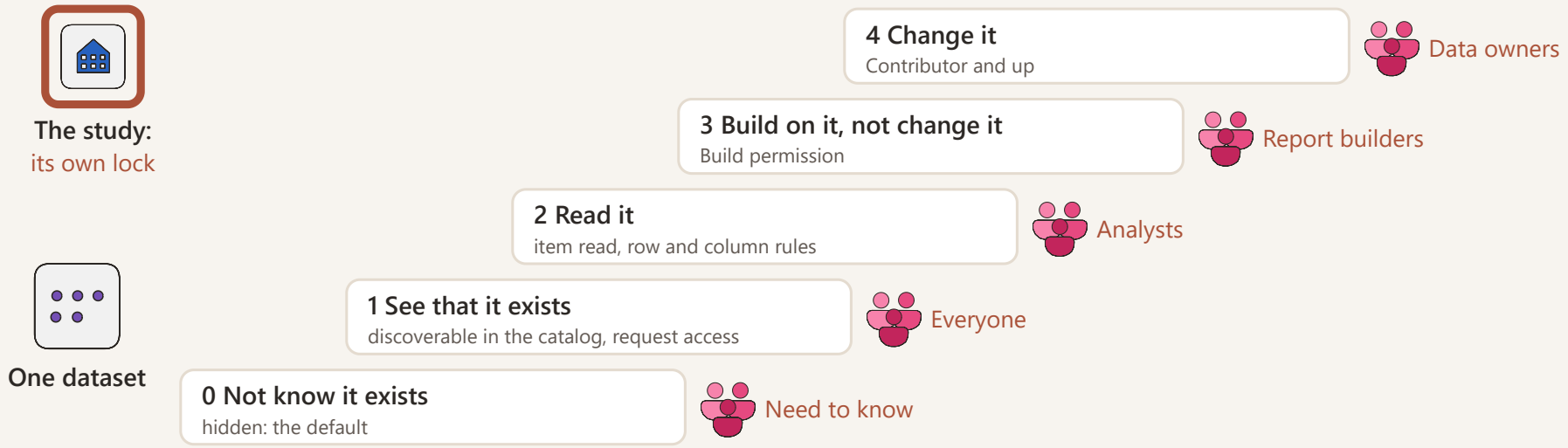


Lock the study, not the kitchen.

Uniformly strict controls on all data drive users to workarounds such as exports and unmanaged storage. Those workarounds are then used for sensitive data too, so over-protecting low-risk data weakens the protection of high-risk data. Classify data and apply the strictest controls where the risk is.

Example: The comic, in Fabric: the study workspace holds the customer data and deserves the lock; the kitchen workspace holds

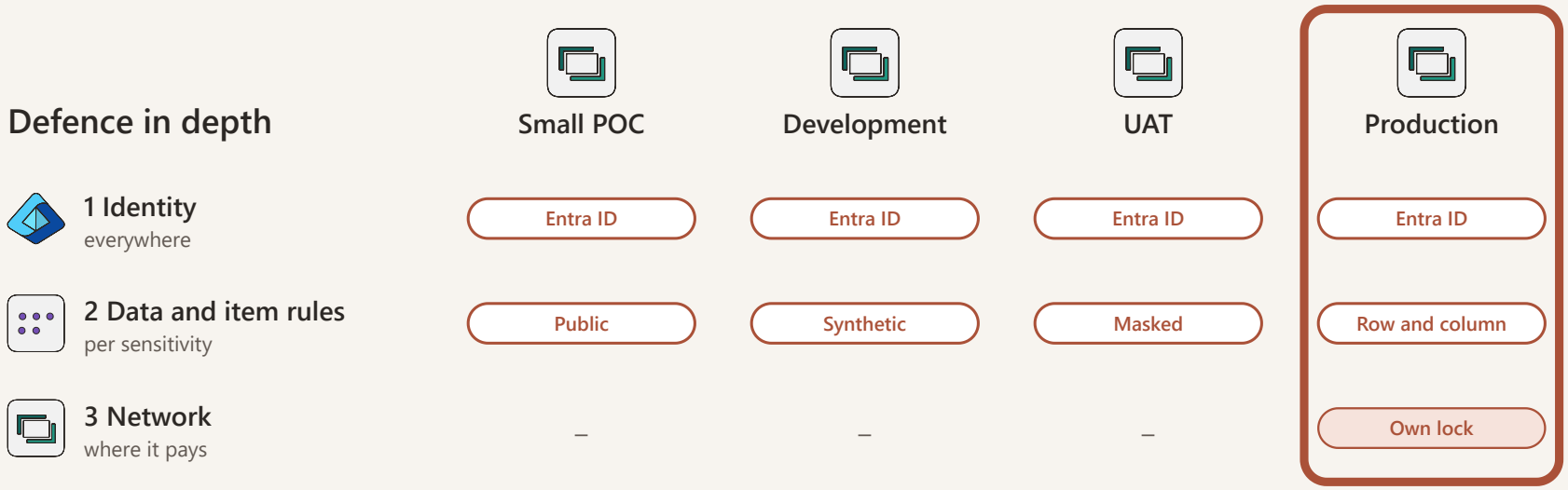
More than open or shut



Hand out the lowest rung that does the job.

Fabric separates not knowing an item exists (the default for items a user cannot access), discovery (endorsed items made discoverable in the OneLake catalog), reading (item permissions, narrowed by OneLake row and column security), building (Build permission on semantic models) and changing (workspace Contributor and above). Workspace Admins, Members and Contributors are not restricted by OneLake row and column security. Network isolation remains available for individual workspaces holding the most sensitive data.

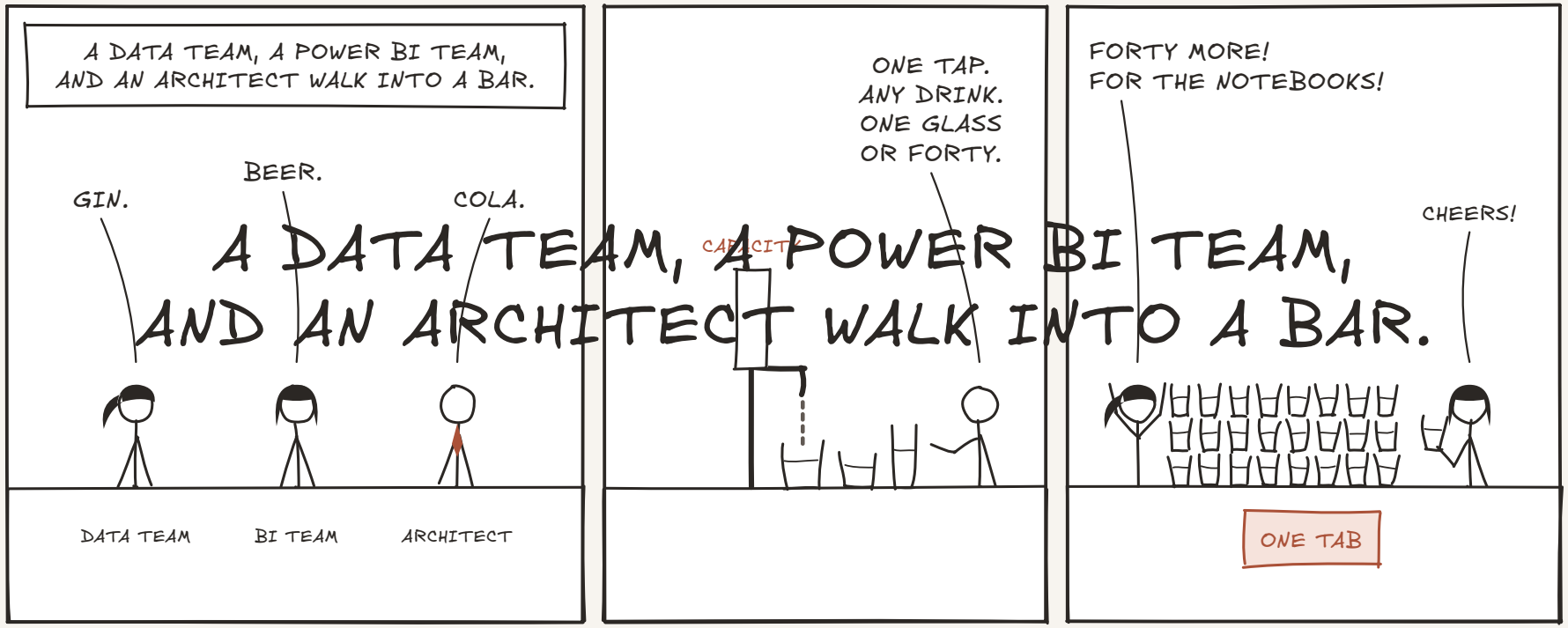
Behind the firewall, what protects the data?



Apply the right measures.

When a network perimeter is the main control, other data protections tend to be neglected, so a breach of the perimeter exposes everything. Starting from identity and data-level access controls, then adding network and other measures in proportion to data sensitivity, gives layered protection without a blanket policy that limits operability.

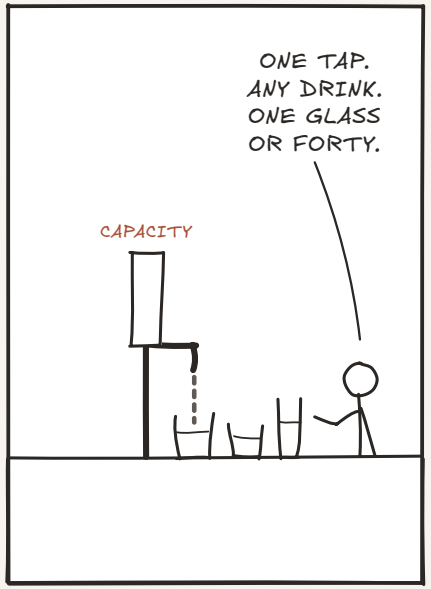
Recap of The lock, delivering the principle of security: 2.1 Security is a shared responsibility. Tune the balance. 2.2 Lock the study, not the kitchen. 2.3 Security is a team sport: people act on what they can see. 2.4 Access is a ladder, not a door.



A capacity is not a server per workload. It is one tap that serves every kind of drink, on one tab, and scales from one glass to forty. Many teams can order different things from the same capacity; the bill and the decisions about it are shared.

Fabric brings Azure data platform teams and Power BI teams onto one tenant and shared capacities. When engineering work lands on a capacity that also serves critical reporting, the resulting dispute is usually about funding and authority rather than

A capacity is not a single server



Capacity

under the hood

- Spark
- Warehouse
- Pipelines
- Power BI

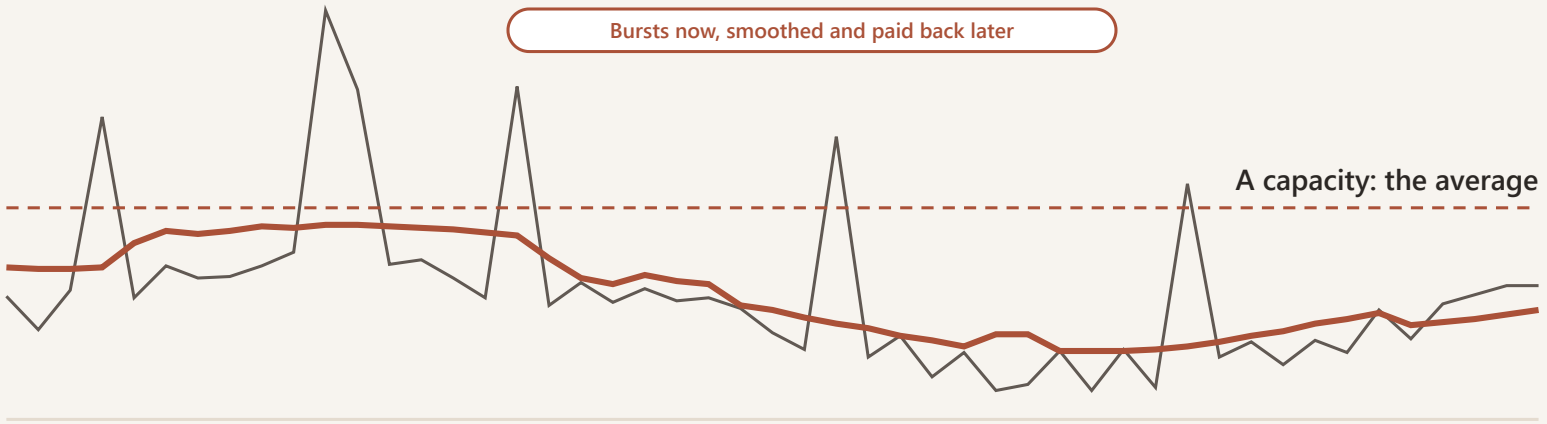
A specialised engine for every kind of order

The capacity is a wrapper for billing: one SKU, one bill, and the guardrails and limits that come with it.

Serverless: a host of specialised machines behind one tap.

A Fabric capacity is not a single server. It wraps a set of engines specialised per workload (Spark, warehouse, data pipelines, Power BI and others) that the platform provisions and runs. The capacity is mainly a billing construct: one purchased SKU, with the guardrails and limits tied to it. From the customer's side the compute is serverless, to a degree that varies per workload.

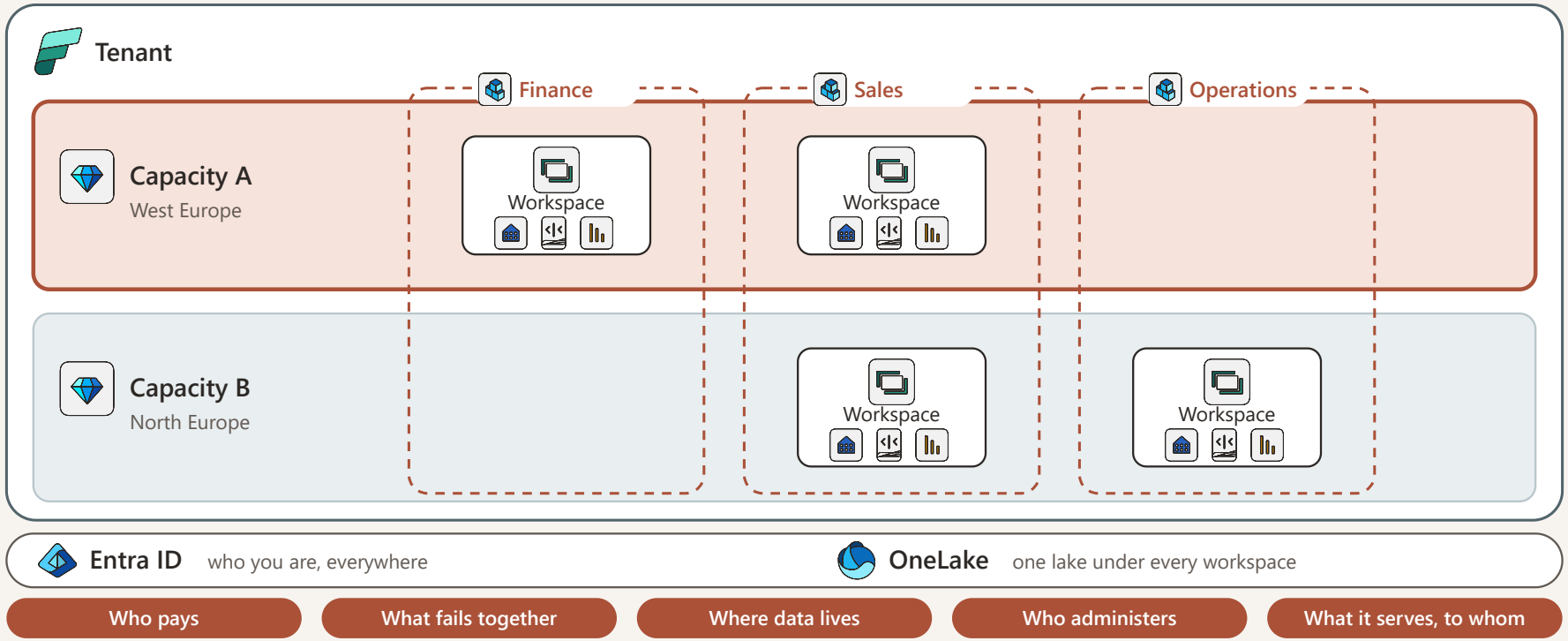
The bachelor party: forty drinks at once



Size for the average, not the peak.

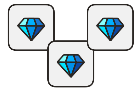
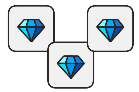
Fabric capacities are not sized like servers. Usage is pooled across all workloads on the capacity, operations may burst above the purchased SKU, and consumption is smoothed over time (background operations over 24 hours, interactive operations over minutes) before throttling applies. Size a capacity on smoothed, combined usage rather than on the peak of any single workload.

Provisioning a capacity: what you decide



All workspaces on a capacity share its compute, its region and its administrators. Provisioning a capacity is therefore a set of decisions: funding, shared failure, data residency, administration, and what may be created on it by whom. These often point to different groupings, which is the design decision to make deliberately.

Pool or split? Central or delegated?

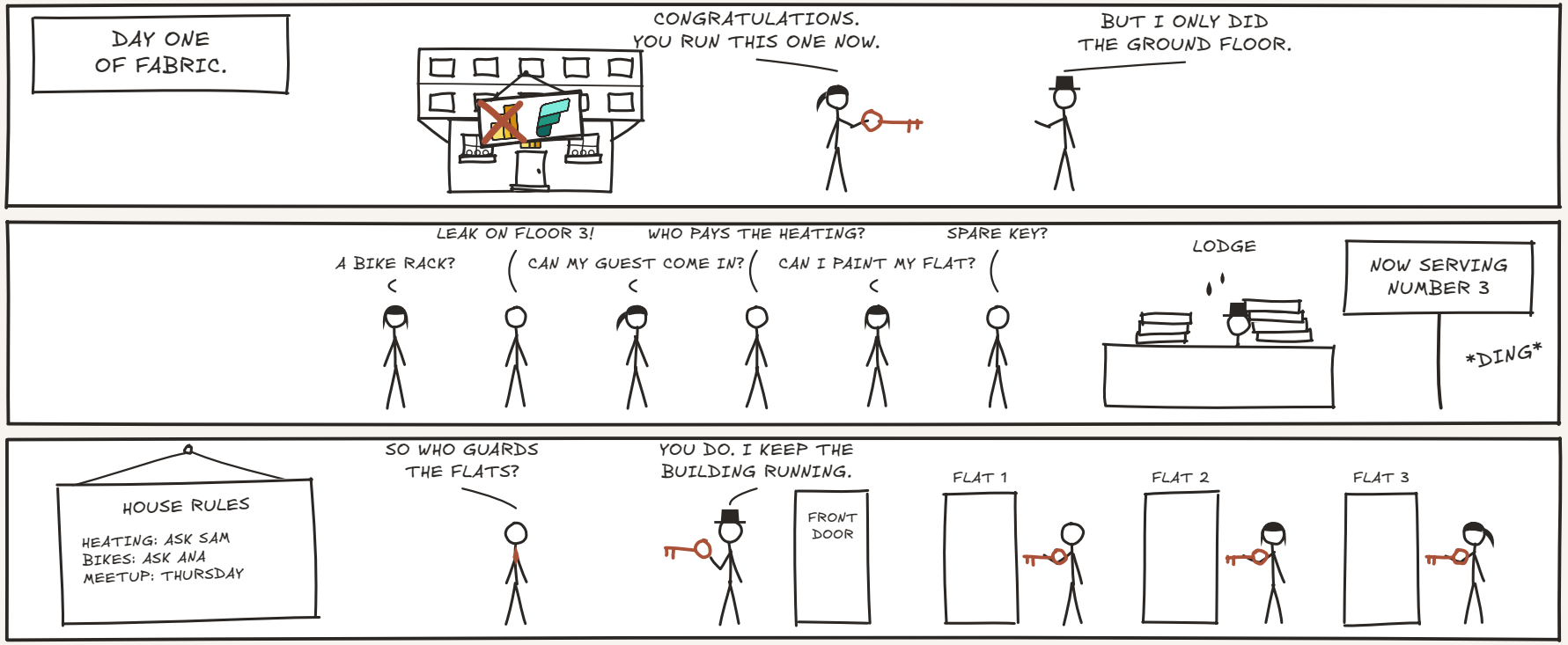
	Pooled one big bar	Split a bar per team
Central the platform team decides	 **One big bar, one manager** Scale, one bill; one queue	 **Many bars, one manager** Isolation; idle bars
Delegated each team decides	 **One big bar, own tabs** Scale; needs chargeback	 **Every team its own bar** Autonomy; duplication

No single answer. Know why you chose yours.

Organisations pool capacities for economies of scale and headroom, or split them for isolation and separate budgets; they centralise capacity decisions for consistency, or delegate them for speed. None of these is right in general. Each has a cost in contention, idle capacity or consistency, so the choice should be made deliberately and its reasons recorded.

Example: Organisations land in every quadrant: pooled for economies of scale, split for isolation, central for consistency,

Recap of The bar, delivering the principle of scalability: 3.1 Size for the average, not the peak. 3.2 Decide what the bar serves, and to whom. 3.3 Pool or split, centralise or delegate: know why.



THE CARETAKER RUNS THE BUILDING. OWNERS HOLD THE KEYS.

A caretaker who looked after a small building finds a much bigger one built on top of it, and is handed the keys. If they keep every key, every resident queues at the lodge with a different kind of question. If they keep the building running while residents lock their own doors, and the noticeboard says who to ask, the building works. A Fabric administrator works best the same way: running the platform, setting house rules and sharing knowledge, while owners control their own data and content.

The same admin. A different job.

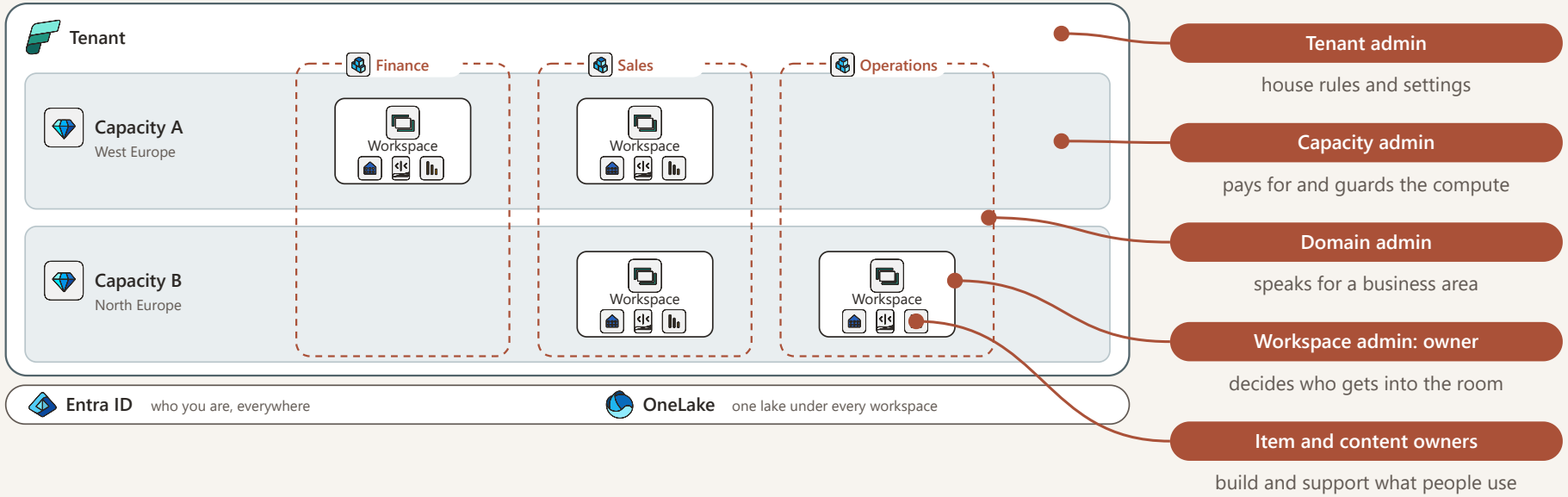


Who can do all five?

Review the role, not just the title.

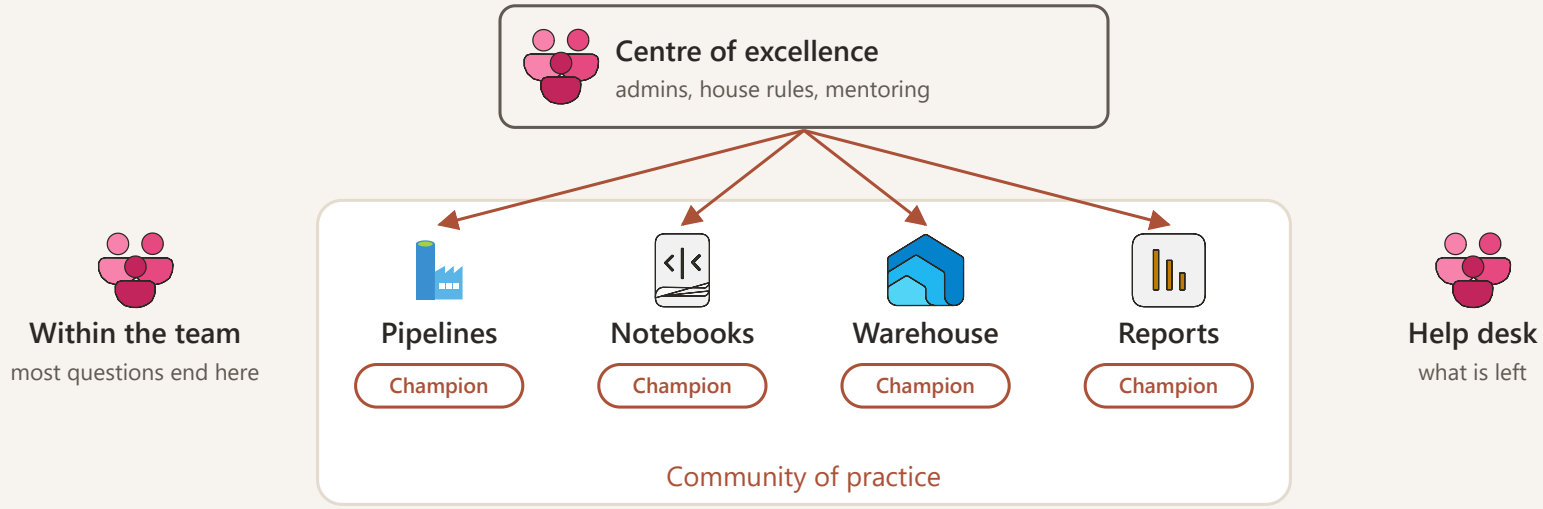
Global administrators and Power Platform administrators are implicitly Fabric administrators, a tenant-wide, high-privilege role. When a Power BI tenant becomes a Fabric tenant, the role covers far more workloads: review who holds it and whether that team is set up to run a data platform. Administrators need broader knowledge, but they should not become the owners of every decision. Microsoft guidance suggests two to four administrators who understand the workloads.

Every layer has an owner



Fabric ownership is layered, from the tenant down to the data: tenant admin, capacity admin, domain admin, workspace admin (in practice the data owner), and item or content owners. Capacity and domain admins are peers: one owns compute, the other a business area.

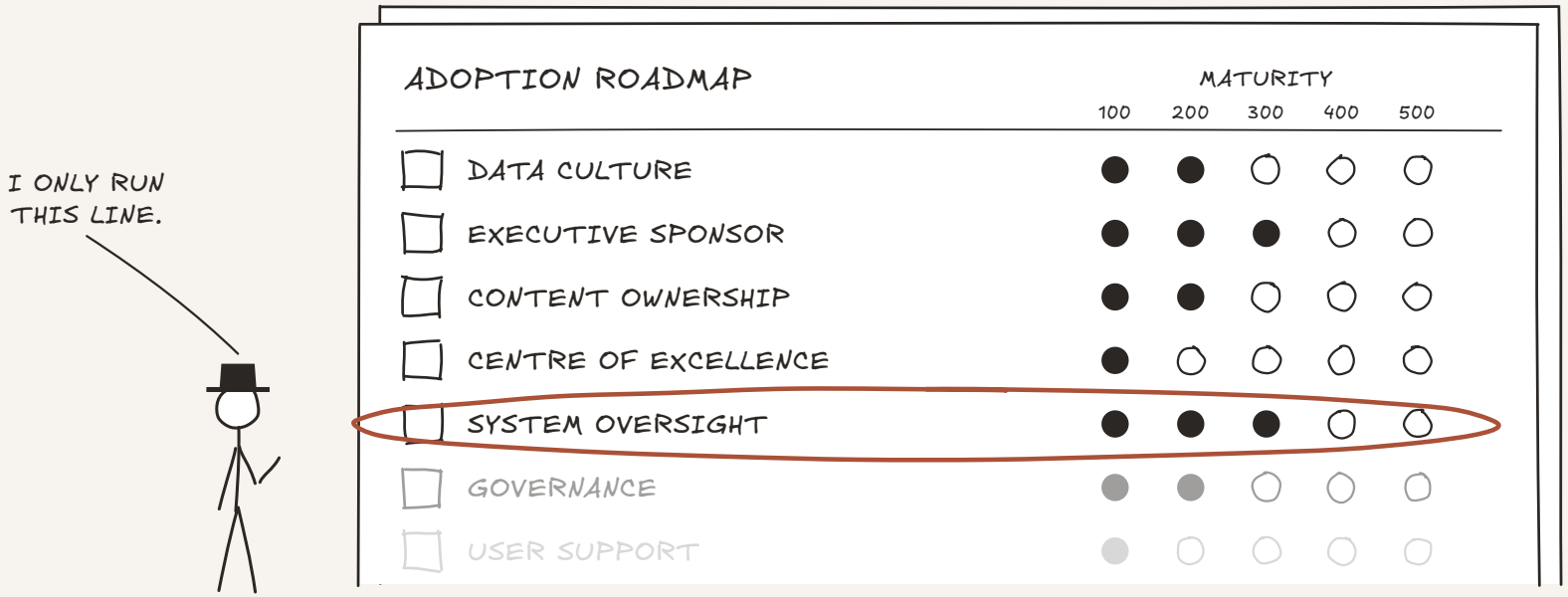
Support in layers, with the centre of excellence in the middle



Spread the knowledge, not the keys.

A Fabric platform covers more workloads than any central team can know in depth. The adoption roadmap describes user support in layers: within the team, through internal champions and a community of practice, through a help desk, and through extended support. Successful organisations combine a centre of excellence, which includes the administrators, with champions in the business who each know a workload and share that knowledge.

The adoption roadmap is a checklist with a maturity scale



The admin runs one area. The organisation runs the roadmap.

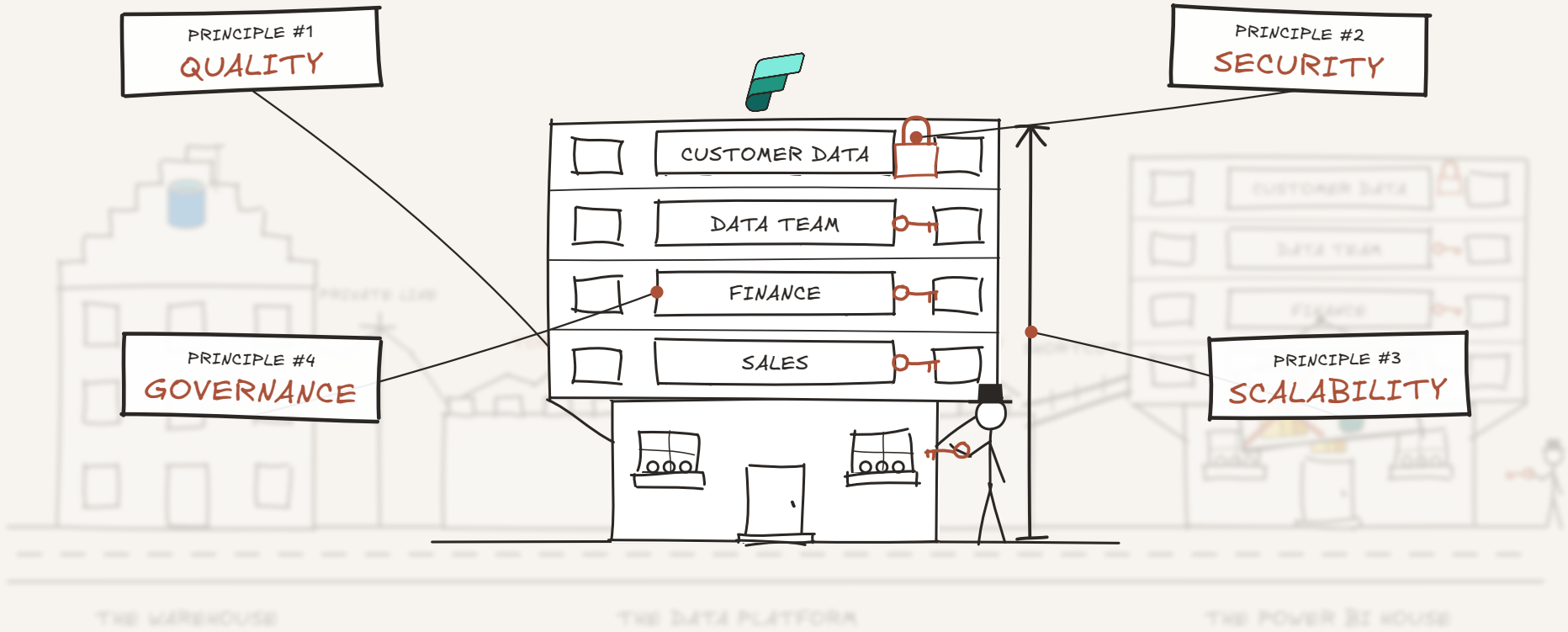
The Microsoft Fabric adoption roadmap describes twelve areas of successful adoption: data culture, executive sponsor, business alignment, content ownership and management, content delivery scope, centre of excellence, governance, mentoring and user enablement, community of practice, user support, system oversight and change management. Administration (system oversight) is one of them. Organisations can assess their maturity per area and improve a few areas at a time. The roadmap is on Microsoft Learn: learn.microsoft.com/power-bi/guidance/fabric-adoption-roadmap

Four stories. One question left: what do you decide on Monday?

Recap of The caretaker, delivering the principle of governance: 4.1 Your Power BI admins just became your data platform admins. 4.2 Ownership has layers. Name an owner for each. 4.3 Support the whole organisation, not just the admin. 4.4 Administration is one area of twelve. The roadmap has the other eleven.

BACK TO THE ESTATE

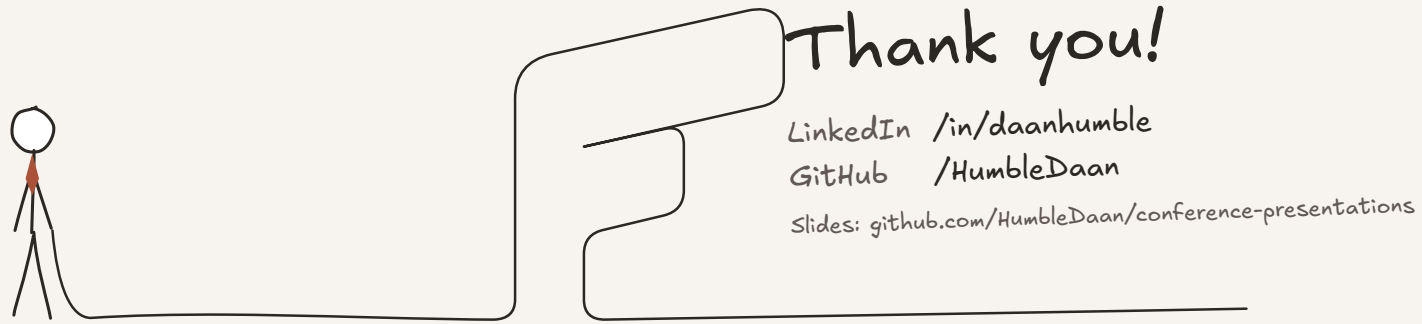
Same estate. Every layer drawn on purpose.



After a migration the estate is the same, but the boundaries the infrastructure used to draw are now drawn on purpose. Quality comes from the move itself: what is connected rather than copied, and what is retired. In the Fabric platform that remains, security protects the most sensitive data specifically, scalability comes from one shared foundation that grows, and governance gives every area a named owner who holds its key, while the platform team keeps the front door. Fabric supports these principles, but they come from organisational decisions, not from the platform.

Your old estate drew your boundaries for you.

In Fabric, you draw them.



Old platforms drew their boundaries through infrastructure: walls, private lines and separate systems. Fabric provides configurable boundaries instead. Deciding where they go, and who owns each one, is the architect's work.

Daan Humblé. LinkedIn: /in/daanhumble. GitHub: /HumbleDaan. Slides: github.com/HumbleDaan/conference-presentations.